lowvram: Transparent GPU Memory Oversubscription for Unmodified CUDA Programs

Henrique Soares
henriquegogo@gmail.com

October 2026 — draft

Abstract

Programs written for a GPU fail with an out-of-memory error as soon as their allocations exceed device memory, even when the host has plenty of free RAM. The established remedies—offloading engines, automatic device placement, custom allocators—all require changes to the program or support in its framework. We present *lowvram*, a 140-line C library that runs unmodified CUDA programs beyond device memory. It serves every `cuMemAlloc` with `cuMemAllocManaged`, so the NVIDIA driver's Unified Memory pages data between GPU and host on demand, and it enlarges the reported device memory so that self-sizing programs use the extra capacity. One ELF file is both the preload library and its launcher (`lowvram python script.py`). On a 6 GiB laptop GPU, lowvram costs nothing measurable when a program fits. On a real workload—the 7.3 GB YuE2-3B music model, which does not load on this GPU—it generates tokens at 43.1 tokens/s, 95% of the speed of a hand-pruned copy of the model that fits in VRAM and 23× faster than Hugging Face Accelerate's offloading, because demand paging keeps resident exactly the half of the weights that token generation uses. When every step touches more data than fits, paging loses: it is 1.4× slower than layer-wise offloading for YuE2's flow-matching step, and on a dense 4B language model it generates 0.58 tokens per second against 1.17 for Accelerate and 3.63 on the CPU. A single driver hint that only the program can give recovers most of that gap. The deciding property is not the size of the program but whether the data a phase reuses fits in VRAM.

1 Introduction

Consumer and laptop GPUs commonly have 4–12 GiB of memory, while the models people want to run—language models, music, image and video generators—often need more. The failure is abrupt: the framework's allocator asks the driver for device memory, the driver refuses, and the program aborts with `CUDA out of memory`, although the machine may have several times that amount of idle system RAM.

The established remedies require intervention in the program. Frameworks offer offloading engines [18, 17, 20] and automatic device placement [8] that splits a model between GPU and CPU; research systems swap tensors under the control of a planner [19, 9, 15]; inference engines run part of a model on the CPU [6, 21, 1]. Each works for the programs written against it, but a user holding an arbitrary research repository cannot easily apply any of them.

CUDA Unified Memory [13] offers a different mechanism. Memory allocated with `cudaMallocManaged` is addressable from CPU and GPU, and on Pascal and later GPUs under Linux it can exceed device memory: the driver migrates pages to the GPU on fault and evicts the least recently used ones when device memory is full [23, 5]. Programs must opt in, however, and mainstream frameworks do not.

lowvram closes that gap without touching the program. It is a shared library, loaded with `LD_PRELOAD`, that turns ordinary device allocations into managed ones at the CUDA driver API. Our contributions are:

- A minimal interposition design (§3) that reaches the CUDA runtime, which resolves driver functions privately through `dlsym` and `cuGetProcAddress` and so escapes plain symbol interposition.
- A single-file packaging technique: one ELF object that is both the preload library and the launcher.
- A measurement study on a 6 GiB laptop GPU (§4–§5) that compares transparent paging with manual streaming, framework offloading, CPU execution and an oracle, on synthetic kernels and on two real models—a 7.3 GB music model with two execution paths and a dense 4B language model—and tests whether driver hints close the gap to hand-written offloading.
- A characterization of where transparent oversubscription is valuable and where it is not (§6).

2 Background: four ways to run beyond VRAM

A device allocation (`cudaMalloc`) is backed by physical GPU memory for its whole lifetime; the driver never pages it out, so there is no fallback when memory runs out. Data that does not fit must live in host RAM, and there are four ways to use it (Table 1). The bandwidths that bound them on our test machine are 300 GB/s for VRAM, roughly 6 GB/s for the PCIe link (§4) and tens of GB/s for host RAM.

Strategy	What happens to data that does not fit	Examples
CPU execution	stays in RAM; the CPU computes on it	PyTorch on CPU, llama.cpp `-ngl 0`
Hybrid split	stays in RAM; the CPU computes those layers or experts	llama.cpp `-ngl N`, KTransformers
Explicit offload	copied to the GPU before use, by a plan	Accelerate, ZeRO-Offload, FlexGen
Demand paging	migrated to the GPU on access, by the driver	Unified Memory; lowvram

Table 1: Strategies for working sets larger than device memory. Only demand paging needs no knowledge of the program.

The first three need knowledge of the computation: which tensors exist, when they are used, where to compute them. Demand paging needs none; its cost is that transfers start only after a fault, in small granules, and that eviction follows recency rather than future use.

3 Design

3.1 Allocation

PyTorch's caching allocator [14], like most CUDA programs, obtains device memory through `cudaMalloc`, which calls the driver's `cuMemAlloc_v2`. lowvram replaces that function:

```
CUresult cuMemAlloc_v2(CUdeviceptr *dptr, size_t size) {
    init();
    return real_alloc_managed(dptr, size, CU_MEM_ATTACH_GLOBAL);
}
```

The returned pointer is a valid device pointer, so kernels and libraries such as cuBLAS and cuDNN use it unchanged, and `cuMemFree` releases it without a hook. While the working set fits,

pages stay resident after their first touch and cost the same as device memory. Beyond that, the driver evicts pages to host memory and faults them back in on access: an out-of-memory failure becomes a slowdown.

3.2 Reported capacity

Some programs size themselves from `cudaMemGetInfo`: automatic device placement, for instance, puts on the CPU whatever does not fit in the reported free memory. lowvram therefore adds half of physical RAM to the free and total values returned by `cuMemGetInfo_v2` and `cuDeviceTotalMem_v2`. The addition is constant: programs that read free memory once behave correctly, but one that allocates until reported free memory is exhausted can exceed physical RAM.

3.3 Reaching the CUDA runtime

Symbol interposition alone does not reach most CUDA programs. The CUDA runtime does not link against `libcuda`: it opens the driver with `dlopen`, obtains `cuGetProcAddress` through `dlsym`, and resolves every other driver function through it. lowvram therefore also defines `dlsym`, `cuGetProcAddress` and `cuGetProcAddress_v2`. Each calls the real implementation and substitutes a hook when the requested name is one lowvram replaces. The two lookup paths use different names—the exported symbol (`cuMemAlloc_v2`) for `dlsym`, the base name (`cuMemAlloc`) for `cuGetProcAddress`—so the substitution table accepts both. The real `dlsym` is obtained with `dlvsym(RTLD_NEXT, "dlsym", ...)` to avoid recursion. Initialization is lazy and happens once, on the first hooked call, because other libraries may call `dlsym` before lowvram's constructors run; without an NVIDIA driver the library is inert, which matters because the preload also reaches launchers and shells.

3.4 Single-file launcher

lowvram is a shared object that is also executable, as glibc's own `libc.so.6` is: a `.interp` section names the dynamic loader, and a linker-specified entry point (`-Wl,-e,lowvram_main`) runs when the file is executed. The entry point reads its arguments from `/proc/self/cmdline` and its own path from `/proc/self/exe`, prepends that path to `LD_PRELOAD` and `execs` the command. Two pitfalls: without C start-up code the entry point receives no `argc`/`argv` and the object's constructors do not run, so the arguments must come from `/proc`; and the entry point is reached without a return address, so it must realign the stack (`force_align_arg_pointer`) and never return. Building a position-independent executable and preloading it instead is not possible: glibc 2.30 and later refuse to load PIE executables as libraries. Because `LD_PRELOAD` is inherited, wrappers and children are covered: `lowvram uv run script.py` reaches the Python interpreter that `uv` starts.

3.5 Scope

Allocations that bypass `cuMemAlloc` are not converted: stream-ordered allocation (`cudaMallocAsync`) and the virtual memory management API (`cuMemCreate`/ `cuMemMap`), which PyTorch uses when `expandable_segments` is enabled. These remain limited to device memory. The implementation targets Linux with glibc on x86-64. [TODO: AArch64: fix the dlvsym fallback (GLIBC_2.17) and test before claiming it.]

4 Methodology

Machine. NVIDIA GeForce RTX 3060 Laptop GPU (GA106, 6 GiB, compute capability 8.6), Intel Core i7-11800H (8 cores, 16 threads), 31 GiB RAM, Linux 7.0, NVIDIA driver 595.91.07,

glibc 2.43, PyTorch 2.14.1 with CUDA 13.0, Transformers 4.57.6. The GPU also drives the display, which holds about 0.7 GiB, so 4.89 GiB were free to the benchmarks. Its link is PCIe 4.0 x16-capable but trained at x8 by the laptop; pinned host-to-device copies reach 6.1 GB/s and device-to-host 6.3 GB/s. This bandwidth, not the GPU, bounds every strategy that moves data.

Procedure. Every configuration runs in a fresh process, with and without lowvram preloaded. Synthetic benchmarks report the median of five timed repetitions after a warm-up (twenty for GEMM, three or two for slow CPU runs); the YuE2 benchmarks repeat each configuration in 3 separate processes and report the median. An out-of-memory error is recorded as a result. The benchmark scripts, the raw JSON results and the script that turns them into every figure, table and number in this paper are in `docs/bench` of the source repository.

Synthetic workloads. (i) *GEMM*: an 8192×8192 matrix product in FP16 and FP32. (ii) *Sweep*: a working set of 256 MiB tensors, each incremented in place once per pass in a fixed order; we report effective bandwidth (bytes read plus written per second). (iii) *Layer stack*: a chain of 8192×8192 FP16 matrix products standing in for transformer inference, with 2–12 GiB of weights and batch 16 (memory-bound) or 2048 (compute-bound). It runs in seven modes: *gpu* (weights allocated on the GPU; unmodified PyTorch or lowvram), *manual streaming* (weights in pinned RAM, each layer copied to the GPU just before use), *overlapped streaming* (the copy of layer $i{+}1$ overlaps the product of layer i on a second stream, the standard double-buffering scheme), *CPU* (FP32, because the CPU lacks fast FP16 arithmetic), and three *hint* modes that run under lowvram and add one driver hint from the program: `READ_MOSTLY` on all weights (evicting a read-duplicated page drops it instead of copying it back), *prefetch* (`cuMemPrefetchAsync` of the next two layers on a side stream), and *remote map* (the layers that do not fit a 3.5 GiB budget are pinned to host RAM and mapped for direct access, so they never migrate). The hint modes measure how much an informed program could recover; lowvram itself issues no hints.

Real workload: YuE2-3B. YuE2 [12] is a music generation model whose 3.6 billion BF16 parameters occupy 7.26 GB (6.76 GiB)—more than this GPU has. It is an AR–NAR "mixture of transformers": each of its 28 layers holds two attention blocks and two MLPs, one pair for autoregressive token generation (AR) and one for a flow-matching decoder (NAR). Token generation uses only the AR pair, the embeddings and the output head, about 3.4 GiB; a flow-matching step runs both pairs over the whole sequence. We measure both phases with the model's own code through Transformers: AR prefill of 512 tokens and greedy decoding of 96 tokens with a KV cache (excluding the first 16 from the median), and NAR velocity evaluations over a 1,538-token sequence (512 context tokens, 1,024 latent frames). Inputs are synthetic token ids and latents of the real shapes, since timing depends on shapes, not values. Modes: unmodified PyTorch; lowvram; Accelerate [8] with `device_map="auto"` and a 4 GiB weight budget, the standard way to run an oversized Hugging Face model; CPU in BF16; and, for token generation, an *oracle* that deletes the NAR half before moving the model to the GPU, so that the AR half runs from ordinary device memory. The oracle requires knowing the model's structure; it is the speed lowvram would reach with perfect placement.

Real workload: a dense language model. YuE2's two paths favor demand paging, so we add the general case: Qwen3-4B, a dense decoder whose 7.5 GiB of BF16 weights are all read for every token. We measure prompt processing of 1,024 tokens (compute-bound) and greedy generation of 48 tokens (memory-bound) through Transformers, in the same modes as YuE2 except the oracle, which does not exist for a model with no idle half.

5 Results

5.1 No cost when the working set fits

Workload	Unmodified	lowvram	Change
GEMM FP16 (TFLOP/s)	26.29	26.29	-0.0%
GEMM FP32 (TFLOP/s)	8.82	8.79	-0.3%
Sweep 1 GiB (GB/s)	309	308	-0.2%
Sweep 4 GiB (GB/s)	301	303	+0.7%
Layers 2 GiB, batch 16 (ms)	7.6	7.4	-2.3%
Layers 4 GiB, batch 16 (ms)	15.6	14.9	-4.7%
Layers 4 GiB, batch 2048 (ms)	351.6	339.0	-3.6%

Table 2: Working sets that fit in VRAM: managed memory performs like device memory.

Table 2 compares unmodified PyTorch with the same program under lowvram when everything fits. GEMM throughput and sweep bandwidth agree within 1%; the layer stack differs by a few percent in both directions, within the spread of the individual repetitions (the ranges overlap; a laptop GPU's clocks vary with temperature). After the first touch, managed pages are resident and accessed at full device bandwidth. lowvram can therefore be left on: it costs nothing until it is needed.

5.2 Cyclic access beyond VRAM

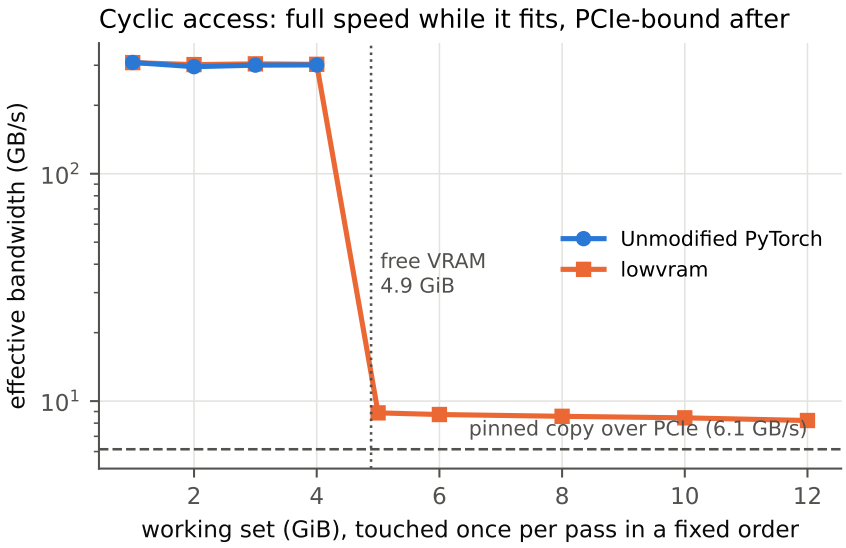


Figure 1: Effective bandwidth of a working set touched once per pass in a fixed order.

Figure 1 shows the cost of oversubscription in its purest form. While the working set fits, both configurations run at about 304 GB/s; unmodified PyTorch fails beyond 4.89 GiB, and lowvram drops to 8.5 GB/s at every larger size. Cyclic access over a set larger than memory is the worst case for least-recently-used eviction: the page needed next is always the one evicted longest ago, so every access misses and each pass migrates the whole set over PCIe. The effective rate sits slightly above the pinned-copy bandwidth because migration runs in both directions at once (pages in, evicted pages out) and the metric counts bytes read plus written. The cliff is the whole story of demand paging: no gradual degradation, but full speed or PCIe speed, depending on whether the set that is *reused* fits.

5.3 Layer stack: paging versus streaming, and what hints recover

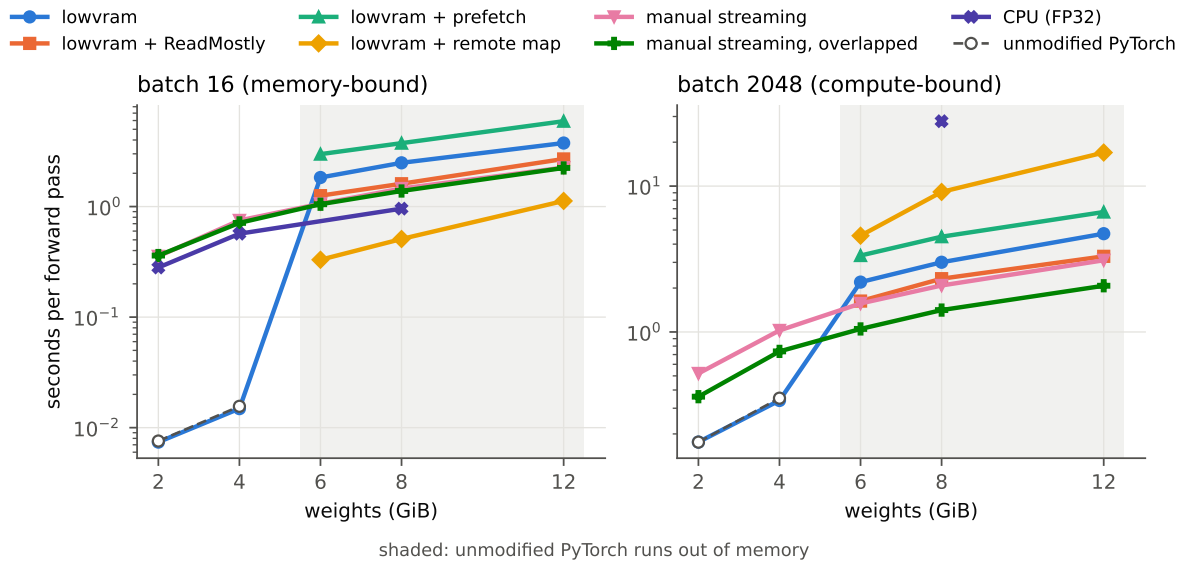


Figure 2: Seconds per forward pass of the layer stack. Unmodified PyTorch fails beyond the free VRAM (shaded).

	batch 16			batch 2048		
Mode	6 GiB	8 GiB	12 GiB	6 GiB	8 GiB	12 GiB
Unmodified	OOM	OOM	OOM	OOM	OOM	OOM
lowvram	1.83	2.49	3.76	2.20	3.01	4.72
lowvram + ReadMostly	1.26	1.61	2.70	1.63	2.32	3.30
lowvram + prefetch	2.98	3.75	5.92	3.34	4.50	6.64
lowvram + remote map	0.33	0.51	1.12	4.57	9.12	16.98
manual streaming	1.06	1.46	2.25	1.57	2.08	3.10
manual streaming, overlapped	1.05	1.38	2.24	1.05	1.41	2.07
CPU (FP32)	–	0.96	–	–	27.85	–

Table 3: Layer stack beyond VRAM, seconds per forward pass (median).

The layer stack reads every weight once per pass, in order: the same cyclic pattern, now with computation in between (Figure 2, Table 3; numbers below are for 8 GiB of weights).

Paging versus streaming. lowvram alone takes 2.49 s per pass at batch 16 and 3.01 s at batch 2048, against 1.46 and 2.08 s for manual streaming and 1.38 and 1.41 s for overlapped streaming. Demand paging is thus 1.4–2.1× slower than a hand-written scheme. Three properties explain the gap: transfers start only after a fault, in small granules, instead of as large DMA copies; they do not overlap computation, because the kernel that needs the data is the one waiting for it; and eviction copies every page back to host memory, although the weights never change, so PCIe carries each byte twice.

What one hint recovers. The hint modes test how much of that gap a program could close by telling the driver one thing it knows. `READ_MOSTLY` on the weights removes the write-back: evicting a read-duplicated page drops it. That alone brings paging to 1.61 and 2.32 s, about 10% slower than manual streaming at batch 16. Prefetching the next two layers makes things *worse* (3.75 and 4.50 s): when memory is full, every prefetch forces an eviction, and the prefetched pages compete with the ones in use. Mapping the layers beyond a 3.5 GiB budget for direct

access from host memory, so that they never migrate, is the fastest configuration of all for the memory-bound case (0.51 s, 2.7× faster than overlapped streaming): the first 3.5 GiB stay resident, and the rest are read once over PCIe, at the link's speed, without copies or faults. The same mapping is the slowest for the compute-bound case (9.12 s), because a large matrix product reads each weight many times, and every one of those reads crosses PCIe.

CPU crossover. At batch 16 the CPU computes the 8 GiB stack in 0.96 s, faster than lowvram, manual streaming and overlapped streaming, although it reads FP32 weights, twice the bytes: it reads host memory directly at tens of GB/s, while every GPU mode is limited by the 6 GB/s link. At batch 2048 the CPU needs 27.85 s, and every GPU mode wins by an order of magnitude.

The lesson for lowvram is that the mechanism is sound but uninformed. Its default—migrate on fault, evict by recency, write back—is the right one for data that is reused and modified, and the wrong one for read-only data streamed once per pass. A hint that only the program can give recovers most of the difference.

5.4 A real model: YuE2-3B

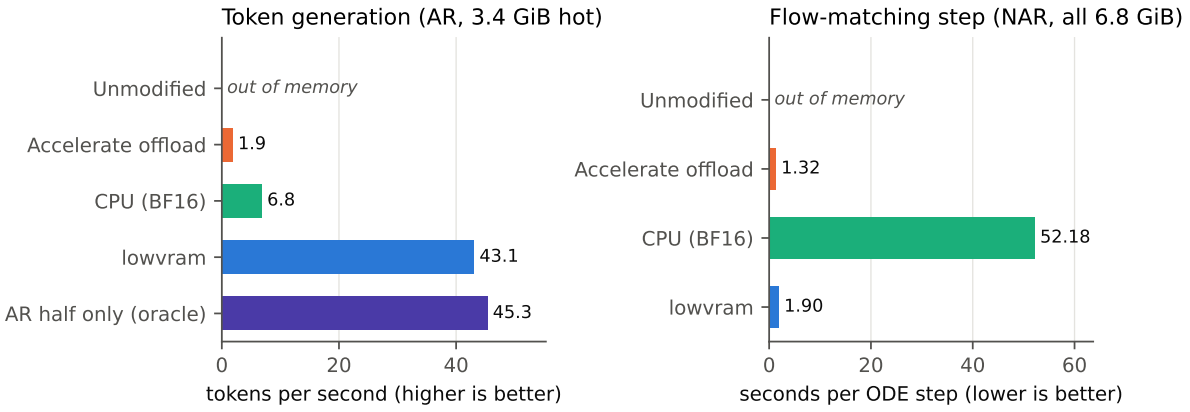


Figure 3: YuE2-3B on a 6 GiB GPU. Left: token generation, which uses half of the weights. Right: a flow-matching step, which uses all of them.

	Token generation (AR)		Flow-matching step (NAR)	
Mode	prompt tok/s	decode tok/s	s/step	GPU weights
Unmodified	OOM	OOM	OOM	all
Accelerate offload	607	1.9	1.32	part
CPU (BF16)	46	6.8	52.18	none
lowvram	611	43.1	1.90	all (paged)
AR half only (oracle)	1694	45.3	–	AR half

Table 4: YuE2-3B, median over 3 runs (CPU: one run).

Unmodified PyTorch cannot load YuE2 on this GPU. Under lowvram the same code generates 43.1 tokens per second (range over runs 41.0–46.0): 95% of the 45.3 tokens per second of the oracle, which runs the AR half from ordinary device memory after we deleted the NAR half by hand. Accelerate, the standard way to run an oversized Hugging Face model, reaches 1.9 tokens per second—lowvram is 23× faster—and the CPU 6.8.

The difference is placement granularity. Accelerate places whole decoder layers, the unit the model declares as unsplittable, and each layer on the CPU side carries its NAR attention and MLP along with the AR ones; for every token it copies all offloaded layers to the GPU, including

the half that token generation never touches. Demand paging needs no declaration: the NAR pages are never accessed during generation, so they are evicted once and stay in host memory, and the 3.4 GiB the AR path reads every token stay resident. Figure 5 shows the transient: the first tokens fault the AR weights in, then the per-token latency settles within a few percent of the oracle's. Prompt processing, which runs once, pays for that first touch: 611 tokens per second against 1694 for the oracle and 607 for Accelerate.

The flow-matching step is the opposite case. It runs both halves over a 1,538-token sequence, so all 6.8 GiB are touched every step, in layer order: the cyclic pattern of Figure 1. lowvram needs 1.90 s per step against 1.32 s for Accelerate, which keeps 4 GiB resident and streams the rest in planned copies; paging is 1.4× slower. The CPU needs 52.2 s per step.

The same program therefore shows both regimes. When the data a phase reuses fits, transparent paging finds it at page granularity, without being told, and runs close to native speed; when it does not, a planned scheme with knowledge of the model wins.

5.5 A dense model: Qwen3-4B

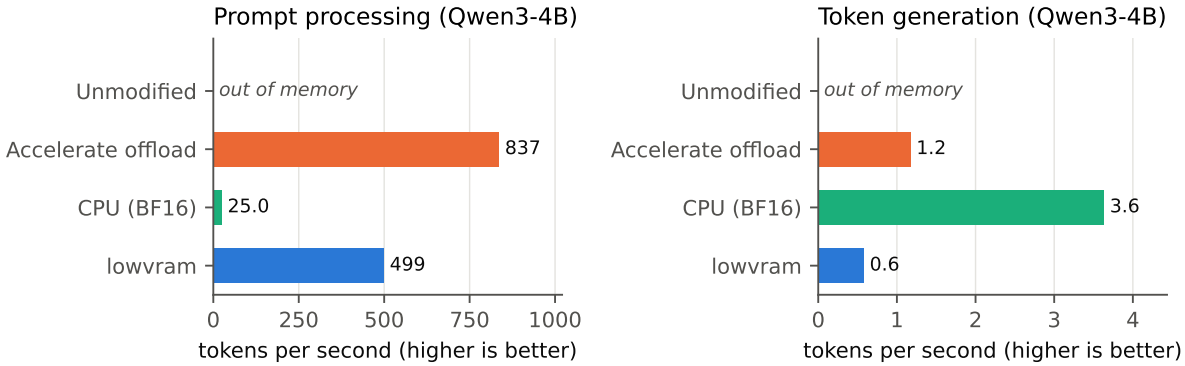


Figure 4: Qwen3-4B (7.5 GiB of BF16 weights) on the same GPU.

YuE2's split hot set favors demand paging. A dense language model is the general case and the unfavorable one: every generated token reads every weight, so the reused set is the whole model. Qwen3-4B holds 7.5 GiB of BF16 weights (Figure 4, Table 5). Unmodified PyTorch cannot load it. Under lowvram, token generation runs at 0.58 tokens per second: each token migrates the model through the 6 GB/s link, in both directions, as in the sweep. Accelerate reaches 1.17 tokens per second by keeping 4 GiB resident and copying only the rest, once, in planned transfers; the CPU reaches 3.63, the fastest of the three, because it reads host memory directly instead of through the link. Prompt processing reuses each weight across the prompt's 1,024 tokens, so transfers matter less: lowvram reaches 499 tokens per second, against 837 for Accelerate and 25 for the CPU. Here lowvram is the slowest GPU option but still an order of magnitude faster than the CPU.

Mode	prompt tok/s	decode tok/s
Unmodified	OOM	OOM
Accelerate offload	837	1.17
CPU (BF16)	25	3.63
lowvram	499	0.58

Table 5: Qwen3-4B, median over 3 runs (CPU: one run).

For dense models larger than VRAM, demand paging is the wrong tool: a planned split is faster in both phases, and for token generation a scheme that computes the remainder on the

CPU—as hybrid engines such as llama.cpp do—is faster still. lowvram turns the failure into a run, but when a framework offers offloading, it should be preferred for such models.

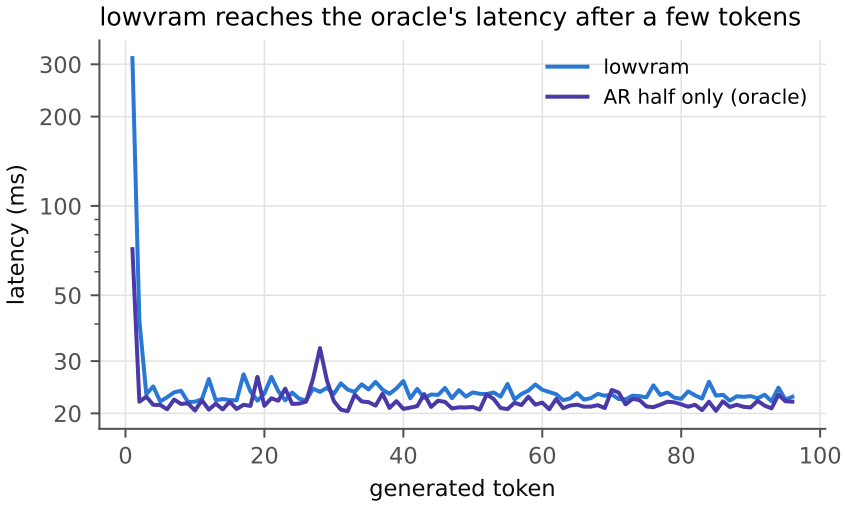


Figure 5: Per-token latency of YuE2 token generation under lowvram and the oracle.

6 Where lowvram is valuable

The measurements reduce to one question: *does the data that a phase reuses fit in VRAM?* The total size of the program does not matter; the reused set does.

Situation	lowvram	Evidence and alternative
Program fits	free	Table 2; can be left on
Total exceeds VRAM, reused set fits (multi-path models, pipelines of several models, rarely used modules)	**best option**	YuE2 generation: 95% of native, 23× Accelerate; no alternative finds the hot set without being told
Transient peak (one large batch, a decode step at high resolution)	**best option**	a crash becomes a slow step; offloading the whole run to avoid one peak costs every step
Reused set exceeds VRAM, compute-bound	works, 1.4–2× slower than planned streaming	YuE2 flow step, layer stack at batch 2048; use framework offloading if available
Reused set exceeds VRAM, memory-bound (dense LLM decoding)	works, but slowest	Qwen3-4B, layer stack at batch 16; CPU or a hybrid engine (llama.cpp) is faster
Multi-process GPU sharing, `cudaMallocAsync`, `cuMemCreate`	not covered	allocations bypass the hook

Table 6: When transparent oversubscription is the right tool.

In practice the comparison is often not with an optimized offload engine but with a program that cannot run at all, or with porting it to a framework that can offload. lowvram makes the program run first; Table 6 tells whether it is then worth the effort to do better.

What could close the gap. The hint experiments point at a middle ground between a transparent shim and a framework. A program—or a framework integration—that marks its

weights `READ_MOSTLY` recovers most of the distance to hand-written streaming, and one that knows which weights are read once per pass can map them for direct access and beat streaming in the memory-bound regime. Neither hint is safe to apply blindly to every allocation: read-mostly pages are expensive to write, and direct access is slow for data that is reused. Applying them from allocation-time heuristics (size, first-write behavior) is future work.

7 Limitations and threats to validity

One machine, one GPU generation and one PCIe configuration (x8); a desktop GPU with an x16 link has twice the transfer bandwidth and would shift every oversubscribed number. The GPU also drove the display. YuE2 inputs are synthetic; the sampling loop is greedy; we do not measure audio quality, which lowvram cannot change. Reported free memory is inflated by a constant, which can lead a program to exhaust host RAM. The comparison with Accelerate uses one weight budget. [TODO: CPU baseline in BF16 on a CPU without native BF16; llama.cpp hybrid not measured.]

8 Related work

Unified Memory. Unified Memory oversubscription and its performance have been studied extensively: Landaverde et al. [11] measured early Unified Memory performance; Zheng et al. [23] examined paged GPU memory and the role of prefetching; Ganguly et al. [5] the interplay of prefetching and eviction under oversubscription; DeepUM [10] adds correlation prefetching for DNN training. TensorFlow exposed oversubscription through a memory fraction above one [22], and the paged optimizers of bitsandbytes [2] place optimizer state in managed memory. lowvram applies the mechanism to unmodified programs, below any framework.

API interposition. Intercepting the CUDA API is how GPUs are remoted and shared: rCUDA [3] and GVirtuS [7] forward calls to remote GPUs, and Kubernetes sharing layers such as HAMi [16] interpose on allocation to enforce quotas. lowvram uses the same technique for a different end, and documents the lookup path a modern CUDA runtime requires.

Tensor swapping and offloading. vDNN [19] offloads activations during training; SwapAdvisor [9] and Capuchin [15] plan swapping and recomputation; ZeRO-Offload and ZeRO-Infinity [18, 17] offload optimizer state and parameters to CPU and NVMe; FlexGen [20] schedules LLM inference across GPU, CPU and disk; Accelerate [8] places modules across devices. These systems use knowledge of the computation to outperform demand paging, but require running within their framework.

Hybrid CPU–GPU inference. llama.cpp [6] computes the layers that do not fit on the CPU; PowerInfer [21] splits hot and cold neurons between GPU and CPU; KTransformers [1] and expert offloading [4] do so for mixture-of-experts models. For memory-bound decoding they beat any scheme that moves weights over PCIe, at the cost of model-specific engines.

9 Conclusion

A small interposition library—three replaced driver functions plus the lookups that reach them—runs unmodified CUDA programs beyond device memory. It costs nothing when a program fits and turns out-of-memory failures into runs when it does not. Its speed beyond VRAM is decided by one property of the workload, whether the data a phase reuses fits: when it does, demand paging finds it at page granularity without being told, and a 7.3 GB music model runs on a

6 GiB laptop GPU at 95% of native speed, 23× faster than standard offloading; when it does not, paging is bound by PCIe and a planned or CPU-based scheme is faster. A single driver hint from the program recovers most of that difference, which marks the next step: moving from a transparent shim towards an informed one, without giving up the property that makes it useful—no change to the program.

References

[1] Hongtao Chen et al. KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2025.

[2] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[3] José Duato, Antonio J. Peña, Federico Silla, Rafael Mayo, and Enrique S. Quintana-Ortí. rCUDA: Reducing the number of GPU-based accelerators in high performance clusters. In *International Conference on High Performance Computing and Simulation (HPCS)*, 2010.

[4] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. arXiv:2312.17238, 2023.

[5] Debashis Ganguly, Ziyu Zhang, Jun Yang, and Rami Melhem. Interplay between hardware prefetcher and page eviction policy in CPU-GPU unified virtual memory. In *International Symposium on Computer Architecture (ISCA)*, 2019.

[6] Georgi Gerganov et al. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, 2023.

[7] Giulio Giunta, Raffaele Montella, Giuseppe Agrillo, and Giuseppe Coviello. A GPGPU transparent virtualization component for high performance computing clouds. In *Euro-Par Parallel Processing*, 2010.

[8] Sylvain Gugger, Lysandre Debut, Thomas Wolf, et al. Accelerate: Training and inference at scale made simple, efficient and adaptable. `https://github.com/huggingface/accelerate`, 2022.

[9] Chien-Chin Huang, Gu Jin, and Jinyang Li. SwapAdvisor: Pushing deep learning beyond the GPU memory limit via smart swapping. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.

[10] Jaehoon Jung, Jinpyo Kim, and Jaejin Lee. DeepUM: Tensor migration and prefetching in unified memory. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[11] Raphael Landaverde, Tiansheng Zhang, Ayse K. Coskun, and Martin Herbordt. An investigation of unified memory access performance in CUDA. In *IEEE High Performance Extreme Computing Conference (HPEC)*, 2014.

[12] Multimodal Art Projection (M-A-P). YuE2-3B: model card and checkpoint. `https://huggingface.co/m-a-p/YuE2-3B`, 2026.

[13] NVIDIA Corporation. CUDA C++ Programming Guide: Unified memory programming. `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`, 2026.

[14] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[15] Xuan Peng, Xuanhua Shi, Hulin Dai, et al. Capuchin: Tensor-based GPU memory management for deep learning. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.

[16] Project HAMi. HAMi: Heterogeneous AI computing virtualization middleware. `https://github.com/Project-HAMi/HAMi`, 2024.

[17] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

[18] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, et al. ZeRO-Offload: Democratizing billion-scale model training. In *USENIX Annual Technical Conference (ATC)*, 2021.

[19] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016.

[20] Ying Sheng, Lianmin Zheng, Binhang Yuan, et al. FlexGen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning (ICML)*, 2023.

[21] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. PowerInfer: Fast large language model serving with a consumer-grade GPU. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2024.

[22] TensorFlow. GPUOptions: per_process_gpu_memory_fraction and experimental unified memory. `https://www.tensorflow.org/api_docs/python/tf/compat/v1/GPUOptions`, 2019.

[23] Tianhao Zheng, David Nellans, Arslan Zulfiqar, Mark Stephenson, and Stephen W. Keckler. Towards high performance paged memory for GPUs. In *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2016.